

PyGameMaker — Tutoriel 15 : Fusion de Fruits — Corrigé

Réservé aux enseignant·e·s ne pas distribuer aux élèves. La numérotation ci-dessous correspond à la Feuille d'exercices de ce tutoriel.

Corrigé de la feuille d'exercices

Partie A : 1-C, 2-D, 3-A, 4-B. **Partie B :** 1. `obj_player`, Collision avec `obj_fruit_cherry`. 2. `obj_spawn_cherry`, Alarme 0. 3. `obj_fruit_cherry` (et `obj_fruit_strawberry`, `obj_fruit_orange`), Hors de la salle. 4. `obj_win_text`, Quand dessiner. 5. `obj_win_text`, Touche pressée : Espace. **Partie C :**

- 1 La branche `*Si*` de la collision vérifie seulement `held_level == 3` (tenir une orange) ; si `held_level` est différent, le `*Si*` est faux et la branche `*Sinon*` s'exécute à la place — le type du fruit lui-même ne change jamais, seul ce que l'on tient change, et une cerise ne peut jamais correspondre qu'à un panier tenant déjà une cerise.
- 2 La fusion en pastèque ne se déclencherait jamais : une capture correspondante ajouterait encore 50 points et changerait le sprite seulement si ces actions étaient (aussi) laissées dans la branche `*Si*`, mais `*Aller à la salle room_win*` placé dans `*Sinon*` se déclencherait à la place à chaque MAUVAISE capture, envoyant le joueur vers la salle de victoire sans jamais vraiment terminer la fusion — l'inverse du comportement voulu.
- 3 Un générateur par type signifie que l'événement de collision pour cet objet précis (`obj_fruit_cherry`, par exemple) identifie déjà le palier, sans conditionnel supplémentaire nécessaire. Un seul générateur aléatoire exigerait que le fruit généré porte sa propre variable « quel palier suis-je », et chaque vérification de collision devrait lire la variable de `other` avant de la comparer à `held_level` — plus de pièces mobiles pour le même résultat.

Parties D et E : réalisation et réflexion.